

Full Stack Developer — Step 2

JavaScript

Roadmap objective: Master JavaScript fundamentals for frontend and backend work

Level: Beginner → Intermediate

Last reviewed: September 2026

How to use these notes

Read the concepts first, reproduce the examples yourself, complete the practical lab, and answer the interview questions without looking at the notes. Use the official documentation links as the final authority for changing APIs and platform behavior.

Learning outcomes

- Variables and scope
- Functions
- Arrays/objects
- DOM
- Events
- Async JavaScript
- Promises
- Fetch
- Modules
- Error handling

Core concepts

1. JavaScript is the bridge between the frontend and Node.js backend in this roadmap.
2. Learn lexical scope, closures, objects, arrays and functions before relying heavily on frameworks.
3. Asynchronous programming uses callbacks, promises and async/await to handle operations that complete later.
4. `fetch()` is commonly used in browsers to call HTTP APIs; always handle network and HTTP-level failures explicitly.

Concept map

```
JavaScript
|
+--> Learn the concept
|
+--> Reproduce a small example
|
+--> Apply it in a feature
|
+--> Test failure/edge cases
|
+--> Document the decision
```

Detailed study guide

1. Variables and scope

Study variables and scope through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

2. Functions

Study functions through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

3. Arrays/objects

Study arrays/objects through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

4. DOM

Study dom through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

5. Events

Study events through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

6. Async JavaScript

Study async javascript through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

Worked example

Async JavaScript

```
async function loadCareer() {
  try {
    const response = await fetch("/api/careers/1");

    if (!response.ok) {
      throw new Error(`HTTP ${response.status}`);
    }

    const career = await response.json();
    return career;
  } catch (error) {
    console.error("Request failed:", error);
  }
}
```

```
}  
}
```

Practice variation: change one input, introduce one failure, predict the result, then run it. Rewrite the example from memory afterward.

Comparison table

Concept	Meaning	Practical use
Promise	Represents future result	then/catch
async/await	Syntax over promises	Readable async flow
Callback	Function passed to another function	Older/common pattern

Common mistakes

- Callback nesting
- Ignoring rejected promises
- No loading/error states
- Global mutable state without need
- Using async code without understanding execution order

Best practices

- Keep responsibilities clear and small.
- Validate inputs at system boundaries.
- Prefer readable code over clever code.
- Test important success and failure paths.
- Use official documentation for current API/platform behavior.
- Keep secrets and credentials out of source control.
- Document important trade-offs so another developer can reproduce your reasoning.

Extended practical lab

Complete these tasks in order. The goal is to turn passive knowledge into evidence that you can actually use the topic.

Lab 1

- Write a synchronous version of a small function.
- Write down what you expected, what happened, and why.

Lab 2

- Convert a network call to Promise-based code.
- Write down what you expected, what happened, and why.

Lab 3

- Rewrite it with `async/await`.
- Write down what you expected, what happened, and why.

Lab 4

- Handle non-2xx responses.
- Write down what you expected, what happened, and why.

Lab 5

- Handle network failure.
- Write down what you expected, what happened, and why.

Lab 6

- Display loading, success and error states in the UI.
- Write down what you expected, what happened, and why.

Mini-project

Build a small dashboard that consumes a public or local API, shows loading/error/success states, and uses `async/await` with clean error handling.

Acceptance criteria

- A new developer can understand the project from its README.
- The main happy path works from start to finish.
- At least three edge/failure cases are tested.
- The project has meaningful Git commits.
- The project includes a short explanation of design choices.
- If deployment applies, production behavior is verified rather than assumed.

Interview preparation

Practice answering these aloud. A strong answer should define the concept, give a concrete example, mention one trade-off, and explain when you would use it.

What is a Promise?

- Answer structure: definition → example → trade-off/limitation → practical use.

async/await vs callbacks?

- Answer structure: definition → example → trade-off/limitation → practical use.

How do you detect an HTTP error with fetch?

- Answer structure: definition → example → trade-off/limitation → practical use.

What is lexical scope?

- Answer structure: definition → example → trade-off/limitation → practical use.

What is a closure?

- Answer structure: definition → example → trade-off/limitation → practical use.

How do you handle loading/error states?

- Answer structure: definition → example → trade-off/limitation → practical use.

Debugging checklist

- Reproduce the problem consistently.
- Reduce it to the smallest failing example.
- Inspect inputs at each boundary.
- Check the first incorrect value, not only the final error.
- Read the complete error message and stack trace.
- Change one thing at a time.
- Retest the original failure after the fix.
- Add a regression test when the bug is important.

Glossary

Term	Your own definition	Example/use
Scope	Write this in your own words.	Give one project example.
Closure	Write this in your own words.	Give one project example.
Promise	Write this in your own words.	Give one project example.
async	Write this in your own words.	Give one project example.
await	Write this in your own words.	Give one project example.
Event loop	Write this in your own words.	Give one project example.
Fetch	Write this in your own words.	Give one project example.
HTTP error	Write this in your own words.	Give one project example.

Term	Your own definition	Example/use
Callback	Write this in your own words.	Give one project example.
Module	Write this in your own words.	Give one project example.

Self-test

- Explain Variables and scope without using its textbook definition.
- Show a small example of Functions.
- What is the most important boundary in this topic?
- What is one failure mode and how would you diagnose it?
- What trade-off should a developer know before using this technique?
- How would you test the normal path and an edge case?
- How does this topic connect to the next roadmap step?
- What would you change if the system had 100× more users/data?
- What part of this topic would you explain differently to a beginner?
- What can you now build that you could not build before studying this step?

Final checklist

- I can explain and demonstrate Variables and scope.
- I can explain and demonstrate Functions.
- I can explain and demonstrate Arrays/objects.
- I can explain and demonstrate DOM.
- I can explain and demonstrate Events.
- I can explain and demonstrate Async JavaScript.
- I can explain and demonstrate Promises.
- I can explain and demonstrate Fetch.
- I can explain and demonstrate Modules.
- I can explain and demonstrate Error handling.
- I completed the practical lab.
- I built or extended the mini-project.
- I tested at least three failure/edge cases.
- I can explain one trade-off.
- I can answer the interview questions without notes.
- I know where to find the authoritative documentation.

Recommended documentation

- <https://developer.mozilla.org/>
- <https://javascript.info/>

Recommended videos

- Modern JavaScript Course (Existing DB resource 98)

Source note: resource references are based on the existing CareerCompass database plus verified web research. For changing APIs, versions, deployment settings and security guidance, consult the linked official documentation before production use.